

SafeReach — Full Project Handoff & Technical Summary

Project Overview

SafeReach is a MERN-stack women safety application designed for realtime emergency SOS assistance.

Core stack:

- React.js
- Node.js
- Express.js
- MongoDB + Mongoose
- Socket.io
- JWT Authentication
- TailwindCSS
- Leaflet Maps
- Firebase Push Notifications

The system has two major user roles:

1. Woman
2. Volunteer

The platform allows women to trigger realtime SOS alerts which are instantly broadcast to nearby active volunteers.

Volunteers can accept SOS alerts, after which a realtime rescue-tracking system becomes active.

Core Functionalities

Authentication System

- JWT-based authentication
 - Role-based authorization
 - Separate woman and volunteer flows
 - Socket authentication using JWT token
-

Volunteer Verification System

Woman Verification

Uses Nyckel AI webcam selfie verification.

Architecture:

- nyckelService.js
 - Service-based integration structure
-

Volunteer Aadhaar Verification

Uses ApyHub API.

Architecture:

- aadhaarService.js
 - Clean service-based external API structure
-

Volunteer Profile System

Volunteer profiles include:

- name
- phone
- bio
- volunteerBio
- availabilityNote
- skills
- languages
- Aadhaar details
- ratings
- totalAlertsHelped
- location

Volunteer profiles are editable.

Profile updates are handled via:

- authController updateProfile
-

SOS Alert System

Alert Lifecycle

Statuses:

- pending
- active
- resolved
- cancelled
- expired

Flow:

1. Woman creates SOS alert
2. Nearby volunteers are notified
3. Volunteer accepts alert
4. Woman receives volunteer details
5. Live tracking begins
6. Woman resolves alert

Alert Model Features

The Alert schema includes:

- woman reference
- volunteer reference
- message
- responseMessage
- geospatial location
- live volunteer location history
- response history
- timestamps
- distance tracking
- review tracking
- emergency type
- priority

Important indexes:

- 2dsphere geospatial index
 - pending alert auto-expiry
 - status indexes
-

Realtime Socket System

Uses Socket.io.

Personal Rooms

Every user joins:

user:<userId>

Example:

user:682b9f

Alert Rooms

Every active SOS gets its own room:

alert:<alertId>

Used for:

- volunteer tracking
 - alert resolution
 - alert cancellation
 - realtime communication
-

Volunteer Rooms

Active volunteers automatically join:

active-volunteers

Used for SOS broadcasting.

Realtime Rescue Tracking System

NEWLY IMPLEMENTED FEATURE

This is the major feature added recently.

When a volunteer accepts an SOS:

The woman receives:

- full volunteer profile details
- phone number
- skills
- languages
- volunteerBio
- availabilityNote
- rating
- current live location

Realtime tracking then begins.

Live Volunteer Tracking

Architecture

Volunteer side continuously broadcasts location.

Woman side receives:

- live volunteer coordinates
 - realtime map updates
 - moving volunteer marker
 - live route line between woman and volunteer
-

Location Broadcasting System

Hook

File:

client/src/hooks/useLocationBroadcast.js

Functionality:

- broadcasts volunteer location every 5 seconds
- uses navigator.geolocation
- emits Socket.io event:

volunteer:location-update

- updates backend DB location
 - linked to specific alertId
-

Socket Events

Volunteer Events

volunteer:toggle-status

Volunteer goes online/offline.

volunteer:location-update

Realtime location update.

Payload:

```
{ coordinates: [lng, lat], alertId }
```

Woman Events

woman:join-alert

Joins alert room.

woman:leave-alert

Leaves alert room.

Alert Events

new-alert

Sent to nearby volunteers.

alert:accepted

Sent when volunteer accepts SOS.

Now includes FULL volunteer profile.

volunteer-location-update

Realtime volunteer coordinates.

alert-resolved

Triggered when woman marks help received.

alert-cancelled

Triggered when SOS is cancelled.

Backend Controller Logic

alertController.js

Important functions:

createAlert()

- validates coordinates
 - prevents duplicate active alerts
 - finds nearby volunteers using MongoDB geo queries
 - broadcasts SOS via Socket.io
 - sends Firebase push notifications
-

acceptAlert()

Major updates implemented:

Now sends:

- full volunteer profile
- volunteer live location
- volunteer metadata
- volunteer skills/languages
- volunteerBio
- volunteer availability

Volunteer joins:

alert:<alertId>

resolveAlert()

- marks alert resolved
 - updates volunteer stats
 - emits alert-resolved
-

Socket Architecture

socket/index.js

Important systems:

JWT Socket Authentication

Socket middleware validates:

- token
- user existence

Attaches user to:

socket.user

Realtime Volunteer Tracking

volunteer:location-update now:

- updates DB location
 - stores history in alert
 - emits live location to woman
 - updates realtime map
-

Frontend Architecture

Context System

AlertContext

Handles:

- active alert
- incoming alerts
- volunteer location
- alert statuses
- realtime socket listeners

Reducer actions:

- ALERT_CREATED
 - ALERT_ACCEPTED
 - ALERT_RESOLVED
 - ALERT_CANCELLED
 - VOLUNTEER_LOCATION
 - RESET
-

SocketContext

Updated recently.

Major improvement:

emit() now supports callbacks.

Before:

```
emit(event, data)
```

Now:

```
emit(event, data, callback)
```

This fixed:

- volunteer accept acknowledgement
- socket callback handling
- async socket confirmations

LiveMap System

Uses:

- react-leaflet
- leaflet
- OpenStreetMap

Features:

- woman marker
 - volunteer marker
 - realtime auto-center
 - fitBounds auto zoom
 - polyline route drawing
 - moving volunteer tracking
-

Volunteer Dashboard

VolunteerHome.jsx

Features:

- realtime SOS feed
- quick response templates
- accept/reject flow
- automatic volunteer tracking activation
- realtime availability state

Major updates:

- activeAcceptedAlert state
 - tracking linked to accepted alert
 - automatic navigation to active alert page
-

Current Realtime Rescue Flow

Complete Flow

1. Woman creates SOS
2. Backend finds nearby volunteers

3. Volunteers receive realtime SOS
 4. Volunteer accepts alert
 5. Woman instantly receives:
 6. volunteer profile
 7. phone
 8. skills
 9. live location
 10. Volunteer location broadcasts every 5 sec
 11. Woman map updates in realtime
 12. Polyline route updates live
 13. Woman resolves SOS
 14. Volunteer stats update
-

Important Technical Notes

Coordinate Format

Backend stores:

[lng, lat]

Leaflet expects:

[lat, lng]

Conversion is necessary on frontend.

Geospatial Search

Uses MongoDB:

`$geoNear`

Search radius:

10km

Maximum volunteers notified:

Push Notifications

Firebase Cloud Messaging used.

Notification triggers:

- new SOS nearby
 - volunteer accepted SOS
-

Current Project State

The project now supports:

- complete SOS lifecycle
 - realtime volunteer dispatch
 - live tracking system
 - volunteer profile forwarding
 - moving map markers
 - realtime route visualization
 - role-based architecture
 - socket-based rescue coordination
-

Recommended Next Features

Potential future upgrades:

- realtime ETA estimation
 - turn-by-turn navigation
 - in-app voice/video calling
 - police integration
 - AI threat analysis
 - panic gesture detection
 - offline SMS fallback
 - rescue timeline playback
 - volunteer route optimization
 - admin dashboard analytics
-

Files Recently Updated

Backend

- controllers/alertController.js
 - socket/index.js
-

Frontend

- client/src/context/SocketContext.jsx
 - client/src/hooks/useLocationBroadcast.js
 - client/src/pages/woman/WomanAlertActive.jsx
 - client/src/components/LiveMap.jsx
 - client/src/pages/volunteer/VolunteerHome.jsx
 - client/src/context/AlertContext.jsx
 - client/src/services/api.js
-

Overall System Design Philosophy

The app follows:

- service-based architecture
- modular socket design
- scalable room-based realtime communication
- clean role separation
- realtime-first emergency response design
- geospatial volunteer dispatching

The architecture is already scalable enough for:

- production deployment
- realtime expansion
- additional emergency services
- multi-city scaling
- advanced tracking features